Towards Dev

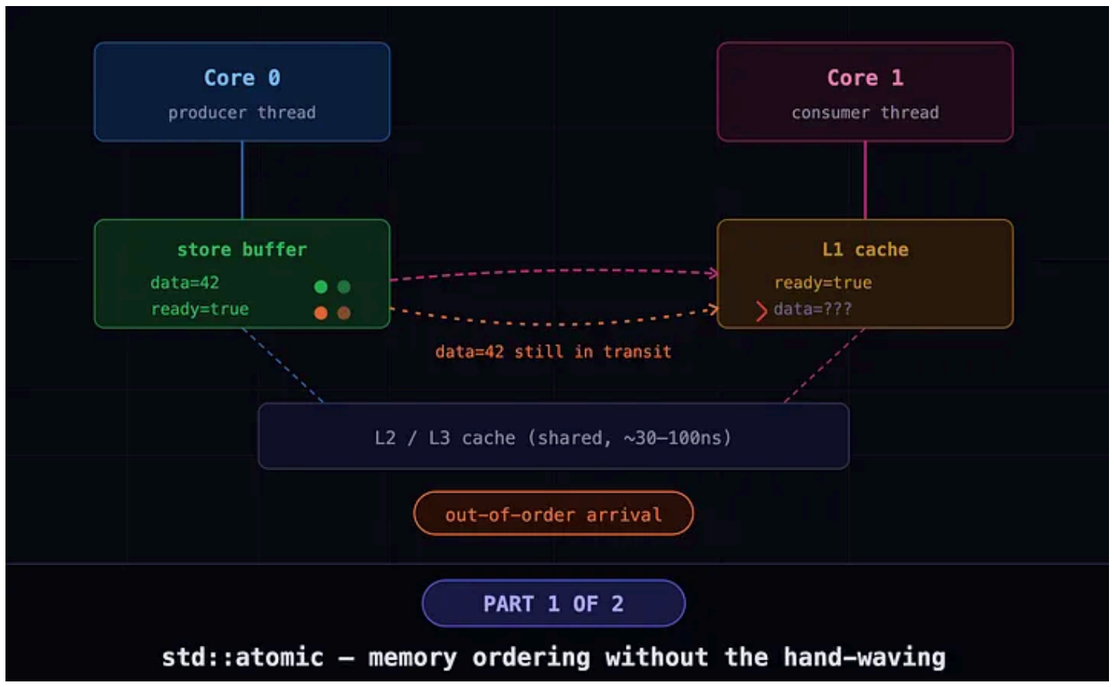
A publication for sharing projects, ideas, codes, and new theories.

Follow publication

`std::atomic`, Explained Properly: Memory Ordering Without the Hand-Waving

Sagar · Following · 11 min read · Mar 23, 2026

4 · 1



Part 1 of 2 | **Audience:** Senior C++ engineers who want depth, not a surface tour

C++11 gave us `std::atomic`. Most people use it wrong — not because they're careless, but because the guarantees are subtler than the syntax suggests. Also because the docs read like they were written for the CPU, not for humans.

If you’re new to concurrency, I recommend starting with:
Atomic Operations, Race Conditions, and Critical Sections — A Clear Introduction

. . .

Why Mutexes Are Not Always the Answer ?

You already know what a mutex does. You lock it, do your work, unlock it. Everything inside the critical section is safe. Simple, honest, boring.

The problem is cost — and two kinds of it.

First: performance.
Even uncontended mutexes aren’t free — they still use atomic instructions, invalidate cache lines, and issue memory barriers. Under contention, they fall back to futex (kernel), bringing scheduler latency and context switches. That turns a cheap lock into a 200ns–µs operation — fine, until it shows up in your profiler at 1am.

Second: composability.
A mutex protects code regions. But what if you just need to increment a counter? That’s one instruction, not a critical section. Using a mutex here is a sledgehammer for a finishing nail — one that occasionally wakes the kernel.

This is what C++11's `std::atomic` is for.

. . .

What `std::atomic` Actually Is ?

`std::atomic<T>` wraps a value of type `T` and guarantees that all operations on it are **indivisible** — no thread will ever observe a partially-written value. For the types the hardware supports natively (1, 2, 4, and 8-byte integers and pointers), this compiles down to a single locked or ordered instruction. No kernel. No mutex. No drama.

```
#include <atomic>

std::atomic<int> counter{0};
// Thread A
counter.fetch_add(1);  // atomic increment - no mutex needed
// Thread B
int val = counter.load();  // atomic read - always sees a complete value
```

Clean. Fast. And here is the thing that trips people up — the thing nobody explains clearly in the first ten articles you read:

Atomicity is not the same as ordering.

The fact that `counter.fetch_add(1)` is indivisible tells you *nothing* about when other threads will see the writes you made *before* that increment. That's a completely separate guarantee. It costs extra. You have to ask for it explicitly.

`std::atomic` gives you both — but only if you know which one you need.

. . .

The Problem `std::atomic` Alone Doesn't Solve

Here's the classic trap. You've probably written something like this:

```
std::atomic<bool> ready{false};
int data = 0;

// Thread A (producer)
void produce() {
    data = 42;          // plain write
    ready.store(true);  // atomic write — this signals Thread B
}

// Thread B (consumer)
void consume() {
    while (!ready.load()) { /* spin */ }
    std::cout << data << "\n";  // is this guaranteed to be 42?
}
```

Your reasoning:

1. Thread B spins until `ready` is `true`.
2. By that point, Thread A has clearly already written `data = 42`.
3. The ordering is obvious from the code."

On x86, you'll be right. Almost always. And *that* is exactly the problem.

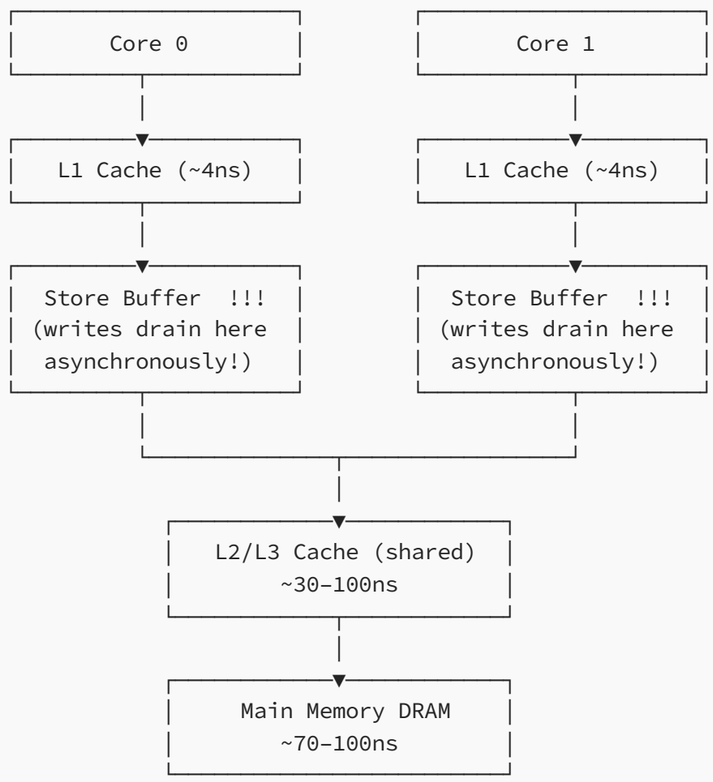
x86 has an unusually strong memory model called **TSO — Total Store Order.** Stores from a single core are observed by other cores in program order. The hardware is quietly fixing your mistake and never telling you. You ship it. It works. You move on.

Then you test on ARM64 — or you deploy to AWS Graviton, or someone runs it on an M-series Mac — and occasionally `data` comes back as `0`. Not a crash.

Not a hang. Just a silent wrong answer. The kind of bug that makes you question your entire career.

Here's why.

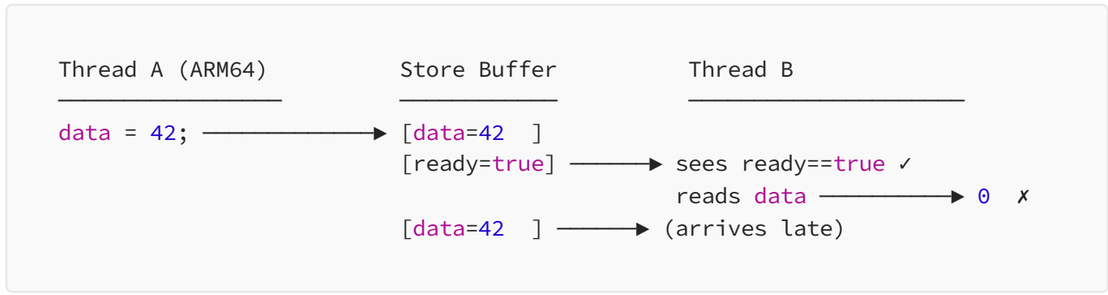
The Hardware Reality: Store Buffers Are the Villain:



When a core writes a value, it goes into the **store buffer** first — not directly into L1. The store buffer drains to L1 asynchronously, in the background, while the core keeps executing. Another core reading that address gets the old value from its own L1 until the write has propagated through the cache coherence protocol (MESI/MOESI).

This is not a bug. It's a feature. Stalling every write until L1 acknowledges it would be catastrophic for throughput. The hardware does this intentionally and the C++ memory model explicitly permits it.

The consequence: the *order* in which writes become visible to other cores can differ from the order in which they were issued.



Both writes left Thread A's core in order. They arrived at Thread B's cache *out* of order. Legal. Permitted. Surprising the first time you see it in a debugger.

This is where `memory_order` comes in.

. . .

Memory Order — What You're Actually Controlling ?

Every `std::atomic` operation takes an optional `std::memory_order` argument. If you don't pass one, you get `memory_order_seq_cst` — the strongest and most expensive option. Most people never change the default, which is safe but unnecessarily slow, a bit like driving everywhere in first gear because you know it won't stall.

There are six values, from weakest to strongest:

```
memory_order_relaxed
memory_order_consume  ← effectively deprecated, compilers just promote it to acq
memory_order_acquire
memory_order_release
memory_order_acq_rel
memory_order_seq_cst
```

Before we go through each one, you need one concept.

Happens-Before:
The entire C++ memory model is defined in terms of **happens-before**. If operation A happens-before operation B, then B is *guaranteed* to observe all side effects of A — every write, fully visible, no exceptions.

Within a single thread this is automatic — `x = 1; y = 2;` means `x = 1` happens-before `y = 2`. No surprises there. The interesting question is how you establish happens-before *across* threads. The answer: through synchronisation operations on atomics. That's the whole game.

. . .

The Six Memory Orders, One by One

`memory_order_relaxed` — Atomicity, and Nothing Else

```
Thread A                    Thread B
_____________               ____________________________

x = 10;
y = 20;

                            a = counter.load(relaxed);
counter.fetch_add(          // a is some value — old or new,
  1, relaxed);              // both are fine.
z = 30;
                            // x, y, z? no idea.
```

```
                                    // could be visible, could not be,
                                    // could arrive in any order.
                                    // no happens-before. nothing.
```

`relaxed` gives you the atomic read or write — indivisible, never torn — and absolutely nothing else. No ordering constraints on surrounding operations. The compiler and CPU can reorder freely around it.

When to use it: Truly independent operations. Hit counters. Diagnostic metrics. Statistics. Frame counters. Any case where you only care that the value was incremented atomically, not about what other memory was visible at the time.

```
// Correct use of relaxed — nobody needs to see anything
// "alongside" this count. It just needs to be accurate eventually.
std::atomic<uint64_t> frames_rendered{0};
frames_rendered.fetch_add(1, std::memory_order_relaxed);
```

When NOT to use it: Any time the atomic is acting as a signal for other data. If another thread reads this value and then reads *something else* based on what it saw — that's a flag, not a counter, and `relaxed` will silently betray you. Usually on ARM64. Usually in production.

. . .

`memory_order_release` + `memory_order_acquire` — The Pair That Does Most of the Work

These two are inseparable. `release` goes on stores. `acquire` goes on loads. Together they form a **synchronises-with** relationship, which is the primary way to establish happens-before across threads. If you understand only one thing from this article, make it these two.

```
Thread A (producer)                    Thread B (consumer)
──────────────────────                 ──────────────────────────────
data = 42;                             while (flag.load(acquire) == false)
more_data = 99;                            { /* spin */ }
flag.store(true, release);

      │                                // happens-before edge established.
      │                                // everything written before
      │                                // the release-store is visible.
      └──── synchronises-with ──────►  assert(data == 42);      // guaranteed
                                       assert(more_data == 99);  // guaranteed
```

The mental model that actually sticks: `release` is sealing a box. Everything you put in before sealing is inside. `acquire` is opening the box. Once you see it's sealed — once the load sees the stored value — everything packed before sealing is visible to you. No extra work needed.

One critical rule that people miss: the synchronises-with only fires if the `acquire`-load actually reads the value written by the `release`-store (or something written after it). If Thread B reads a stale value — one stored before the `release` — the synchronisation does not happen. The flag has to have been actually raised, not just almost raised.

```
std::atomic<bool> flag{false};
int data = 0;
```

```cpp
// Thread A — now correct
void produce() {
    data = 42;
    flag.store(true, std::memory_order_release);  // seal the box
}

// Thread B — now correct
void consume() {
    while (!flag.load(std::memory_order_acquire)) { /* spin */ }
    // synchronises-with established. data is 42. guaranteed.
    std::cout << data << "\n";
}
```

Two words. One on each side. That's it. That's the fix for the bug we looked at earlier.

The quick rule:

- Writing something that signals "data is ready"? `release` on the store.
- Reading something to check "is data ready"? `acquire` on the load.
- `acquire` on a store or `release` on a load? Compile error. The types protect you.

. . .

`memory_order_acq_rel` — For When You're in the Middle

`acq_rel` is for **read-modify-write operations** — `fetch_add`, `fetch_sub`, `exchange`, `compare_exchange_*`. It acts as both an acquire and a release at the same time.

You need this when a thread sits in the *middle* of a chain: it reads a value (needs to acquire what was written before) and writes a new one (needs to release its own writes to what comes after). The canonical example is reference counting:

```
// Shared resource with manual ref count.
// The fetch_sub needs acq_rel because:
//   acquire: see all writes from threads that previously held a ref
//   release: make our writes visible before the count hits zero

std::atomic<int> ref_count{1};

void release_ref() {
    if (ref_count.fetch_sub(1, std::memory_order_acq_rel) == 1) {
        // We were the last one. fetch_sub returned 1, now it's 0.
        // The acquire half ensures we see everything the other
        // ref-holders wrote before they released.
        destroy_resource();
    }
}
```

Use `release` only? The last thread can't see writes from earlier holders. Use `acquire` only? Your writes aren't visible to threads inspecting the count. You need both — hence `acq_rel`.

One constraint worth knowing: **`acq_rel` is only valid on RMW operations.** You can't slap it on a plain store or load. The compiler will tell you, loudly.

. . .

`memory_order_seq_cst` — The Default, and What It Actually Costs

`seq_cst` is the strongest option. It gives you everything `acq_rel` provides, plus one additional guarantee: there is a **single total order** of all `seq_cst` operations across all threads that every thread agrees on.

Without `seq_cst`, `acquire`/`release` only gives you *pairwise* happens-before between specific producers and consumers. Two different thread pairs can observe synchronisations in inconsistent relative orders. `seq_cst` eliminates that: every thread sees all `seq_cst` operations in the same global sequence.

```
// This pattern requires seq_cst to be correct.
// With only acq/rel, the assertions can fire.
std::atomic<bool> x{false}, y{false};
std::atomic<int>  z{0};

// Thread 1        Thread 2        Thread 3              Thread 4
x.store(true);     y.store(true);  if (x.load()) {       if (y.load()) {
                                       if (!y.load())        if (!x.load())
                                           ++z;                  ++z;
                                   }                     }

// With seq_cst: z is 0, 1, or 2. Predictable.
// With acq/rel only: threads 3 and 4 can observe x and y stores
// in different orders, making z == 2 possible when it shouldn't be.
```

In practice, most code never needs this. Straightforward producer/consumer? `release`/`acquire` is correct and cheaper. `seq_cst` is for the rare case where you have *multiple* atomic variables and need all threads to agree on their relative ordering.

The hardware cost is real. On x86, a `seq_cst` store emits `XCHG` or `MOV` + `MFENCE`, which flushes the store buffer. On ARM64, it wraps the instruction in `dmb ish` barriers on both sides. Concretely:

	x86-64	ARM64
relaxed store	MOV	STR
release store	MOV	STLR (one-sided barrier)
seq_cst store	XCHG / MFENCE	DMB ISH + STLR + DMB ISH

That `DMB ISH` on ARM64 drains the *entire* store buffer. In a tight loop it costs 4–8× more than a `release` store. On x86 the gap is smaller, but `MFENCE` still serialises the pipeline.

The rule: Default to `release`/`acquire`. Escalate to `seq_cst` only when you have multiple atomics and can articulate why `acq_rel` isn't sufficient — and write that explanation in a comment, because your future self will not remember.

. . .

The Full API: Load, Store, and RMW

Now that the ordering model is clear, here is the surface you'll use day to day.

Load and Store :

```
std::atomic<int> a{0};

a.store(42);                                    // seq_cst by default
a.store(42, std::memory_order_release);         // cheaper on ARM64

int val = a.load();                             // seq_cst by default
int val = a.load(std::memory_order_acquire);    // cheaper on ARM64
int val = a.load(std::memory_order_relaxed);    // cheapest — no ordering
```

`store` accepts only `relaxed`, `release`, or `seq_cst`.

`load` accepts only `relaxed`, `acquire`, or `seq_cst`.

Passing `acq_rel` to a plain store, or `release` to a plain load, is undefined behaviour — though most compilers catch it before it becomes your problem at runtime.

Read-Modify-Write Operations :

These are the workhorses. All of them return the **old value** — the value *before* the operation ran.

```
std::atomic<int> n{10};

int old = n.fetch_add(5);    // old=10, n is now 15
int old = n.fetch_sub(3);    // old=15, n is now 12
int old = n.fetch_and(0xFF); // bitwise AND, returns old value
int old = n.fetch_or(0x01);  // bitwise OR
int old = n.fetch_xor(0x0F); // bitwise XOR

// All of them take a memory_order
n.fetch_add(1, std::memory_order_relaxed);   // for counters
n.fetch_sub(1, std::memory_order_acq_rel);   // for ref counts
```

> *There are also operator overloads — `++`, `--`, `+=`, `-=`, `&=`, `|=`, `^=` — but they silently use `seq_cst`. They're fine for non-hot paths, but in performance-sensitive code the explicit `fetch_*` methods make the ordering visible and reviewable. Use those.*

`exchange :`

Atomically writes a new value and returns the old one. Useful for spin-locks and ownership transfers:

```
std::atomic<bool> lock{false};

// Attempt to acquire a spin-lock
bool already_locked = lock.exchange(true, std::memory_order_acquire);
// already_locked == false → we got it
// already_locked == true  → someone else has it, try again
```

. . .

The Decision Tree

When you reach for an atomic, run through this before picking a memory order:

```
Does this atomic coordinate access to OTHER data?
│
├── No  → memory_order_relaxed
│         (counters, metrics, independent values)
│
└── Yes → Am I writing to signal "data is ready"?
          │
          ├── Yes → memory_order_release on the store
          │
          └── Am I reading to check "is data ready"?
                │
                ├── Yes → memory_order_acquire on the load
                │
                └── Am I doing RMW in the middle of a chain?
                      │
                      ├── Yes → memory_order_acq_rel
                      │
                      └── Do I need ALL threads to agree on the
                          global order of multiple atomics?
                                │
                                └── Yes → memory_order_seq_cst
                                          (and write a comment explaining why)
```

The vast majority of lock-free code lives in the top three branches. If you find yourself reaching for `seq_cst` in performance-sensitive code, that's a flag to pause and articulate *exactly* why `acq_rel` is insufficient. Usually you'll find it isn't — and you just saved yourself a `MFENCE` on every hot-path iteration.

. . .

You now have the complete mental model: what the hardware actually does, what the C++ memory model lets you express, and how to choose the right ordering constraint for a given operation.

That’s the foundation. In Part 2, we build on it — because there’s a class of problems where simple loads and stores aren’t enough, where multiple